

Projet SETR

Analyse et modélisation selon la méthode SART

Contents

1	Introduction	2
2	Modélisation SART	2
2.1	Diagramme de Contexte	2
2.2	Actigramme (DFD Niveau 0)	2
2.3	Dictionnaire des Données	3
2.4	DFD Niveau 1 — Carte Centrale (ProjectSETR)	3
2.5	DFD Niveau 1 — Carte Distante (ProjectSETR2)	4
3	Spécification des Tâches	4
4	Architecture Matérielle	5
5	Architecture Logicielle	5
5.1	Frameworks et arborescence	5
5.2	Protocole Inter-cartes	5
5.3	Gestion de l'affichage OLED	5
5.4	Protocole Firebase	5
6	Analyse Temporelle	6
6.1	Contraintes et ordonnancement	6
6.2	Gestion des accès concurrents	6
7	Solutions Retenues	6
8	Résultats Obtenus	6
9	Conclusion	7

1 Introduction

Ce rapport présente l'analyse et la modélisation du système embarqué SETR distribué sur deux cartes ESP32 communiquant par liaison série UART. Il couvre la modélisation SART, l'architecture matérielle et logicielle, l'analyse temporelle ainsi que les solutions retenues et les résultats obtenus.

Fonctions assurées :

1. Mesure température/humidité (DHT22) et CO₂ par deux voies UART+PWM (MH-Z19B)
2. Détection de présence ultrason (HC-SR04)
3. Affichage OLED 128x64, dashboard web temps réel + API JSON
4. Upload périodique vers Firebase Realtime Database

2 Modélisation SART

La méthode SART (Structured Analysis for Real-Time systems) distingue :

- **Le modèle environnemental** : diagramme de contexte et actigramme (DFD niveau 0)
- **Le modèle comportemental** : DFD de niveau 1 et spécification des processus

2.1 Diagramme de Contexte

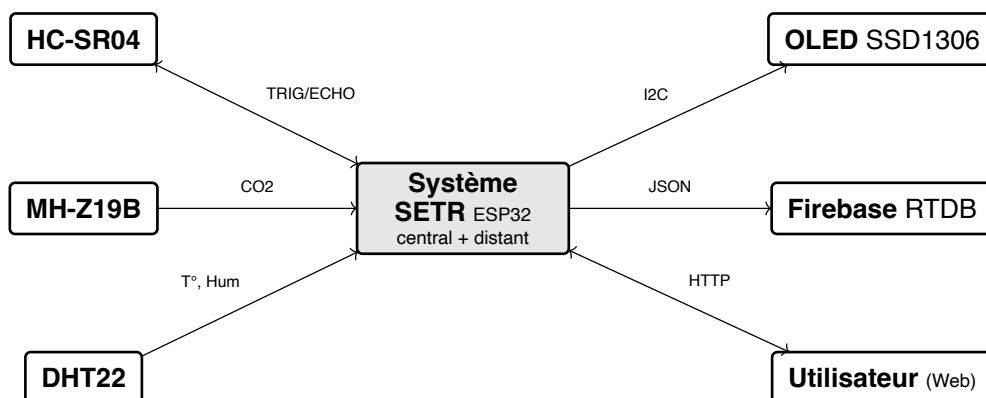


Figure 1: Diagramme de contexte du système SETR

Entité	Flux entrant	Flux sortant
DHT22	—	Température, Humidité
MH-Z19B	—	CO2 UART, CO2 PWM
HC-SR04	Impulsion TRIG	Impulsion ECHO
Utilisateur (Web)	Requêtes HTTP	Page HTML, JSON API
Firebase RTDB	—	Données PATCH/PUT JSON
OLED SSD1306	—	Données I2C (affichage)

Table 1: Flux échangés avec l'environnement

2.2 Actigramme (DFD Niveau 0)

Le système est décomposé en quatre processus. Les capteurs distants (MH-Z19B, HC-SR04) sont acquis par l'ESP32 déporté puis transmis à la carte centrale via RS232 (processus 2, flux interne).

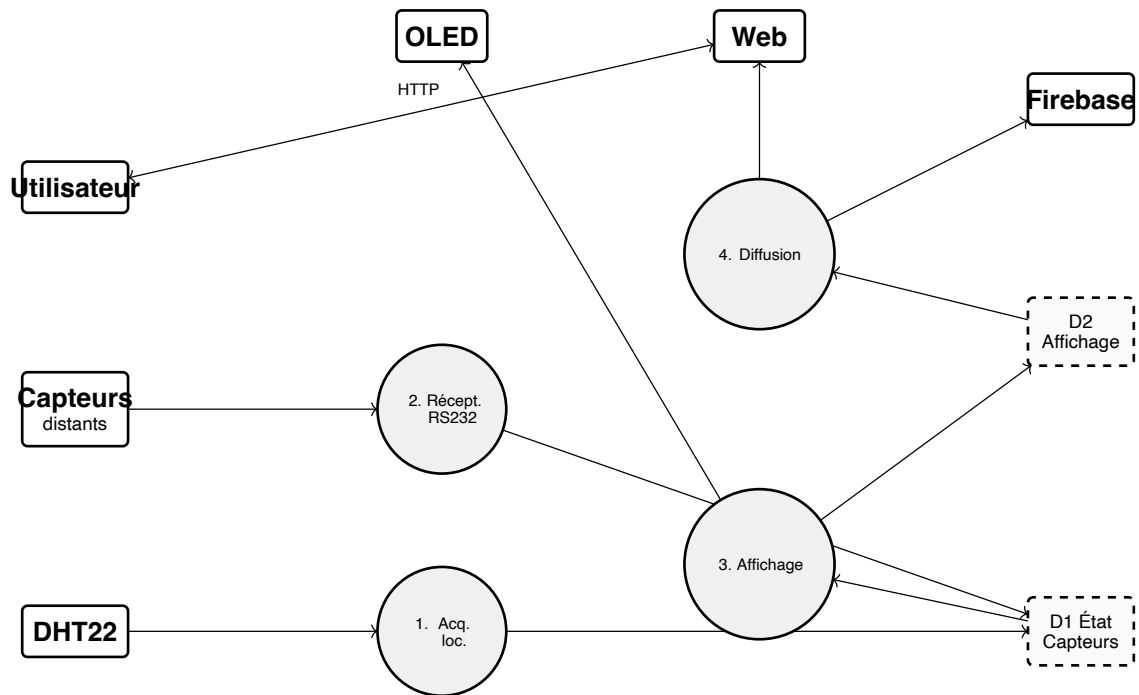


Figure 2: Actigramme (DFD niveau 0)

2.3 Dictionnaire des Données

Flux	Description	Type
Température	Mesure ambiante	float °C
Humidité	Humidité relative	float %
CO2_UART	Concentration CO2 via UART	int ppm
CO2_PWM	Concentration CO2 via PWM	int ppm
Présence	Détection ultrason	bool
Trame RS232	Trame ASCII "CO2_UART:....;CO2_PWM:....;PRES:..."	string
Page Web	Dashboard HTML responsive	string
API JSON	Données temps réel	string
Données Firebase	Payload JSON vers RTDB	string

Table 2: Dictionnaire des flux de données

Stock	Contenu	Structure
État Capteurs (D1)	Dernières valeurs des capteurs	SensorData
Données Affichage (D2)	Lignes de texte pour OLED	DisplayData

Table 3: Stocks de données

```

struct SensorData {
    float temperature;        // °C
    float humidity;           // %
    float co2_ppm_uart;       // ppm
    float co2_ppm_pwm;        // ppm
    bool presence_detected;    // true/false
    unsigned long last_update_dht;
    unsigned long last_update_remote;
};
struct SensorMessage {
    SensorMessageType type;    // Dht ou Remote
    float value1, value2;      // données capteur
    bool flag;                 // présence
};

```

Listing 1: Structures de données

2.4 DFD Niveau 1 — Carte Centrale (ProjectSETR)

Quatre tâches FreeRTOS interconnectées par une file de messages SensorQueue (16 éléments).

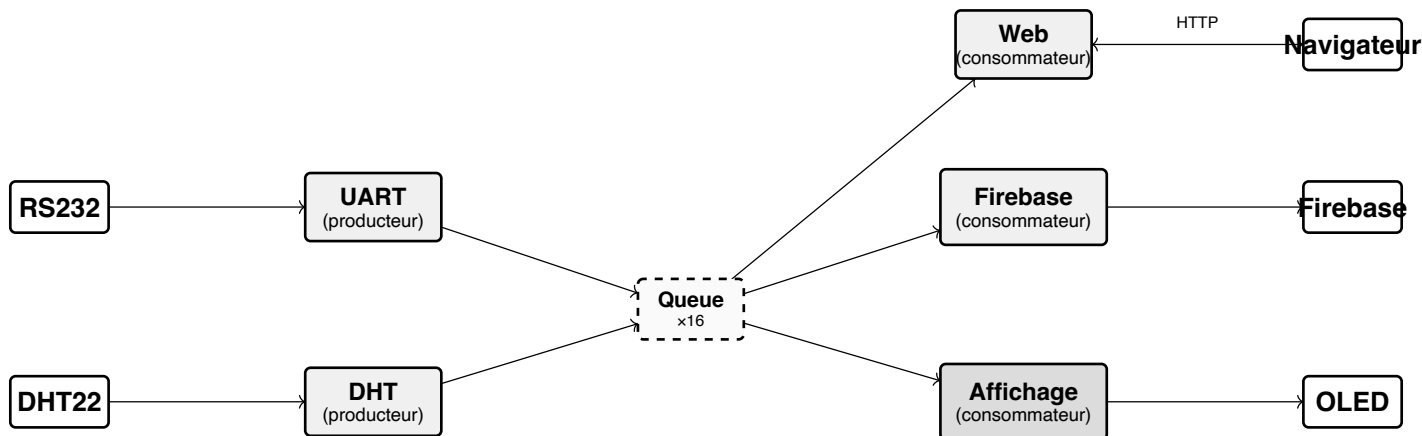


Figure 3: DFD niveau 1 : architecture multitâche de la carte centrale

2.5 DFD Niveau 1 — Carte Distante (ProjectSETR2)

Trois tâches FreeRTOS en pattern producteur/consommateur. Les mesures CO2 (UART+PWM) et présence sont produites périodiquement, consommées par la tâche `uart_consumer` qui formate et émet la trame sur UART2.

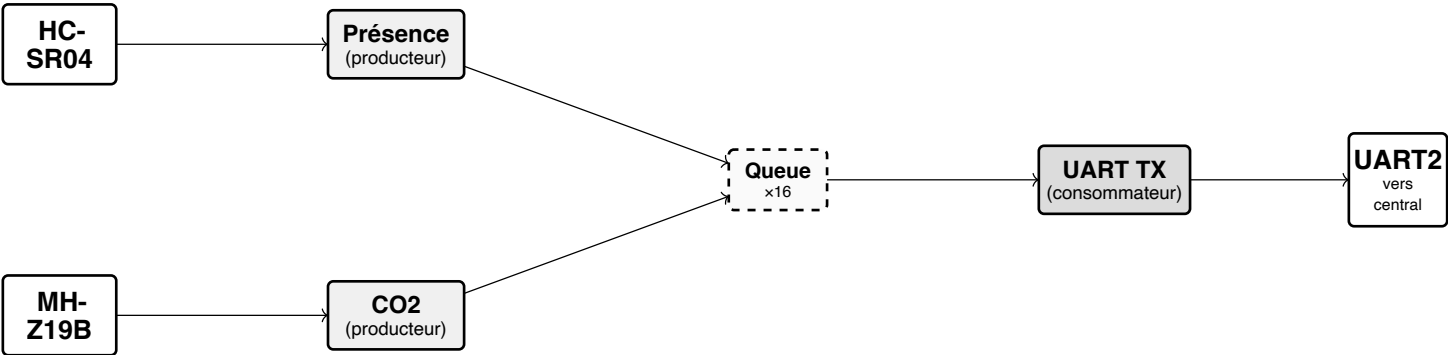


Figure 4: DFD niveau 1 : architecture multitâche de la carte distante

3 Spécification des Tâches

Tâche	Carte	Pile	Cœur	Prio.	Période
DhtProducer	Centrale	4096	1	1	5 s
UartProducer	Centrale	4096	1	1	10 ms (poll)
DisplayConsumer	Centrale	8192	1	2	33 ms
FirebaseUploader	Centrale	8192	0	1	5 s / 60 s
WebDashboard	Centrale	8192	0	1	Apériodique
co2_producer	Distante	3072	—	5	4 s
presence_producer	Distante	4096	—	5	500 ms
uart_consumer	Distante	4096	—	6	4 s

Table 4: Spécification des tâches FreeRTOS

Périodiques : DhtProducer, co2_producer, presence_producer. **Quasi-périodiques** : DisplayConsumer, FirebaseUploader, uart_consumer (activés par file/timer). **Apériodique** : WebDashboard (requêtes HTTP). **Polling** : UartProducer (scrutation buffer UART).

4 Architecture Matérielle

Composant	Broche	Protocole
DHT22	GPIO15	OneWire
SSD1306 OLED	SDA=GPIO5, SCL=GPIO4	I2C (0x3C)
RS232 RX/TX	GPIO26/27 (UART2)	UART 9600 8N1

Table 5: Carte Centrale (ProjectSETR)

Composant	Broche	Protocole
MH-Z19B UART	TX=GPIO16, RX=GPIO17	UART1 9600 8N1
MH-Z19B PWM	GPIO23	PWM (5000 ppm)
HC-SR04	TRIG=GPIO33, ECHO=GPIO32	GPIO out/in
RS232 TX/RX	GPIO27/26 (UART2)	UART 9600 8N1

Table 6: Carte Distante (ProjectSETR2)

5 Architecture Logicielle

5.1 Frameworks et arborescence

La carte centrale utilise le **framework Arduino** (bibliothèques Adafruit DHT22, SSD1306, GFX). La carte distante utilise le **framework ESP-IDF** (C bare-metal) pour un contrôle fin des GPIO/UART et une empreinte mémoire réduite. Les deux projets sont gérés sous PlatformIO.

ProjectSETR/ (Arduino)

```
src/
  main.cpp           - Setup + tâches
  sensor_manager.cpp - DHT22 + parsing RS232
  sensor_data.cpp    - Accès thread-safe
  display_manager.cpp - Pilotage OLED
  display_data.cpp   - Formatage affichage
  global_data.cpp    - Mutex + instances
  firebase_uploader.cpp - Upload WiFi
  web_dashboard.cpp  - Serveur HTTP + API
include/             - Headers + config.h
platformio.ini
```

ProjectSETR2/ (ESP-IDF)

```
src/
  main.c             - Init + tâches
  co2_sensor.c        - MH-Z19B (UART+PWM)
  presence_sensor.c   - HC-SR04
  uart_link.c         - Émission trame UART2
  consumer_task.c     - Queue -> UART TX
include/             - Headers + config.h
platformio.ini
```

5.2 Protocole Inter-cartes

Communication UART à 9600 bauds. Format de trame ASCII :

CO2_UART:<ppm>;CO2_PWM:<ppm>;PRES:<0|1>\n

Le parser de la carte centrale (sensor_manager.cpp) tolère un préfixe de log avant CO2_UART:, ce qui permet le débogage sur la même ligne série.

5.3 Gestion de l'affichage OLED

L'OLED s'éteint automatiquement en l'absence de présence (économie d'énergie). Un pin de synchro externe (GPIO25, macro USE_EXTERNAL_SYNC) permet un pilotage depuis la carte distante. Sans données RS232, l'écran affiche un message d'attente avec les données locales uniquement.

Un mode simulation (SIMULATE_REMOTE_ESP) génère des données CO2/présence factices pour tester la chaîne complète sans la carte distante physique.

5.4 Protocole Firebase

Données envoyées au Firebase Realtime Database via HTTP direct (sans le SDK Firebase, plus léger de 500 Ko) :

1. **Courant** : HTTP PATCH toutes les 5 secondes
2. **Historique** : HTTP PUT toutes les 60 secondes (tampon tournant de 60 slots)

La reconnexion WiFi est automatique (intervalle de 5 s entre tentatives).

6 Analyse Temporelle

6.1 Contraintes et ordonnancement

Contrainte	Valeur	Critique
Échantillonnage température/humidité	5000 ms	Non
Échantillonnage CO2 (UART+PWM)	4000 ms	Non
Échantillonnage présence	500 ms	Oui
Rafraîchissement OLED	33 ms (30 FPS)	Non
Envoi Firebase courant	5000 ms	Non
Envoi Firebase historique	60000 ms	Non
Réponse serveur web	< 100 ms	Non
File de messages	16 éléments	—

Table 7: Contraintes temps réel

Tâches ordonnancées par FreeRTOS en préemptif à priorité fixe :

1. Affichage (priorité 2) servi avant les producteurs (priorité 1)
2. Sur la carte distante, `uart_consumer` (priorité 6) vidange la file avant les producteurs (priorité 5)
3. L'upload Firebase est isolé sur le cœur 0 pour ne pas perturber les tâches temps réel du cœur 1

6.2 Gestion des accès concurrents

Accès aux données partagées protégés par **mutex FreeRTOS** : `sensors_mutex` (SensorData) et `display_mutex` (lignes OLED). Le pattern producteur/consommateur via **file de messages** (16 éléments) découple les tâches d'acquisition du traitement, évitant tout blocage direct.

7 Solutions Retenues

Architecture 2 ESP32 : un seul ESP32 ne dispose pas d'assez d'UART pour le MH-Z19B (UART1), le terminal (UART0) et la liaison inter-cartes. La distribution permet aussi de placer les capteurs dans une zone distincte de l'affichage.

Liaison UART ASCII : native sur ESP32, sans bibliothèque externe. 9600 bauds suffisent pour 40 octets toutes les 4 secondes. Le format ASCII facilite le débogage.

Pattern producteur/consommateur : la Queue FreeRTOS de 16 éléments découple temporellement les acquisitions périodiques du traitement. Capacité équivalente à 80 s de mesures CO2.

Double voie CO2 : lecture simultanée UART + PWM du MH-Z19B pour redondance et fiabilité accrue.

Frameworks : Arduino pour la centrale (bibliothèques Adafruit), ESP-IDF pour la distante (contrôle bas niveau, empreinte réduite).

HTTP direct Firebase : plus léger que le SDK Firebase (500 Ko). Suffisant pour des données RTDB publiques.

Mode simulation intégré : `SIMULATE_REMOTE_ESP` accélère les itérations de développement sans matériel distant.

8 Résultats Obtenus

Toutes les fonctions du cahier des charges sont implémentées et testées :

1. DHT22 : température/humidité toutes les 5 s, tolérance aux erreurs (`isnan`)
2. MH-Z19B : CO2 UART+PWM toutes les 4 s, calibration zero et ABC configurables
3. HC-SR04 : distance toutes les 500 ms, seuil de présence à 50 cm
4. OLED : 4 lignes (T°, H%, CO2 PWM, CO2 UART), extinction automatique, mode attente
5. Dashboard web : page responsive, 5 indicateurs avec âge, rafraîchissement auto 3 s, API JSON `/api/data`
6. Firebase : upload courant (5 s) et historique (60 s), reconnexion WiFi automatique

Les contraintes temps réel sont respectées : la scrutation de présence (500 ms) est garantie, l'affichage 30 FPS est découplé, et l'upload Firebase (cœur 0) ne perturbe pas les tâches critiques.

Limitations : lectures DHT22 occasionnellement invalides (2-3%), upload Firebase non authentifié (données publiques uniquement), test RS232 de bout en bout nécessite le matériel physique (Wokwi ne simule pas la liaison inter-ESP32).

9 Conclusion

Le système SETR met en œuvre une architecture temps réel distribuée sur deux ESP32. La modélisation SART a formalisé les interactions capteurs/sorties (modèle environnemental), la décomposition multitâche FreeRTOS (modèle comportemental) et les structures de données partagées. L'architecture producteur/consommateur avec file de messages et mutex assure le découplage temporel et l'intégrité des données. Les contraintes critiques (présence à 500 ms, trames RS232 toutes les 4 s) sont respectées.